24.2 Hash Code

We face the problem that not every object in Java can easily be converted to a number. However, the key idea behind hashing is the transformation of any object into a numeric representation. The key is to have a hashing function transform our keys into different values, and convert that number into an index to then access the array.

We achieve this through our own implementation of a `hashCode()` function, with a return value of an `int` type. This `int` type is our *hash value*.

The built-in String class in Java, for example, might have the following code block:

```
public class String {
    public int hashCode() {
        //implementation here
    }
}
```

Based on this example, we can call `key.hashCode()` to generate an integer hash code for a `String` instance called `key`.

Memory Inefficiency in Hash Codes

An issue mentioned earlier is memory inefficiency: for a small range of hash values, we can get away with an array that individuates each hash value. That is, every index in the array would represent a unique hash value. This works well if our indices are small and close to zero. But remember that Java's 32-bit integer type can support numbers anywhere between -2,147,483,648 and 2,147,483,647. Now, most of the time, our data won't use anywhere near that many values. But even if we only wanted to support special characters, our array would still need to be 1,112,064 elements long!

Instead, we'll slightly modify our indexing strategy. Let's say we only want to support an array of length 10 so as to avoid allocating excessive amounts of memory. How can we turn a number that is potentially millions or billions large into a value between 0 and 9, inclusive?

Wrapping!

The **modulus operator (%)** allows us to achieve this.

Review: The result of the modulo operator is like a remainder in fractional division. For example, `65 % 10` returns `5` because after dividing 65 by 10, we are left with a remainder of 5. Thus as other examples, `3 % 10 = 3`, `20 % 10 = 0`, and `19 % 10 = 9`. For an intuitive understanding of this, think about how we used the modulo operator in Project 1: Deques to ensure you avoided `IndexOutOfBounds` Exceptions and could accurately index into your deque via the concept of "wrapping around" the array.

Returning back to our original problem, we want to be able to convert any number to a value between 0 and 9, inclusive. Given our discussion on the modulo operator, we can see that any number mod 10 will return an integer value between 0 and 9. This is what we need to index into an array of size 10!

More generally, we can locate the correct index for any key with the following:

```
Math.floorMod(key.hashCode(), array.length)
```

where `array` is the underlying array representing our hash table.

Quick Note: In Java, the `Math.floorMod` function will perform the modulus operation while correctly accounting for negative integers, whereas `%` does not.

24.3 “Valid” & “Good” Hashcodes

Valid Hashcodes!

You may see this term in discussions and potentially on exams. What exactly makes a hash code “valid”? There are two properties:

1. ***Deterministic*:** The `hashCode()` function of two objects A and B who are equal to each other (`A.equals(B) == true`) have the same hashcode. *This also means the hash function cannot rely on attributes of the object that are not reflected in the* `.equals()` *method.*
2. ***Consistent*:** The `hashCode()` function returns the same integer every time it is called on the same instance of an object. This means the `hashCode()` function must be independent of time/stopwatches, random number generators, or any methods that would not give us a consistent `hashCode()` across multiple `hashCode()` function calls on the same object instance.

Note that there are no requirements that state that unequal objects should have different hash function values.

One could argue that these two requirements are in fact the same requirement. We can restate the requirement of consistency. Imagine we make a pointer named `A` to an object `O` at 12:00 pm and a pointer named `B` to this same object `O` at 1:00 pm. We know that the hash code should return the same integer for both objects, due to the consistency requirement. However, how do we formally define our statement “this same object `O`” above? Technically, the only reason we consider `B` to be pointing to the same thing as `A` is because of the `.equals()`method! This is starting to sound a lot like the determinism requirement.

Good Hashcodes!

You’ll probably see this term a lot as well. But what makes a hashcode “good”? There are a few properties that can make a good `hashCode()`:

1. The `hashCode()` function must be valid.
2. The `hashCode()` function values should be spread as uniformly as possible over the set of all integers.
3. The `hashCode()` function should be relatively quick to compute [ideally O(1) constant time mathematical operations]

Now let’s think more specifically about the impact of the hashing function. In general, we assume most hash functions will be “relatively quick”. Why do we make this assumption? Given how intrinsic the hashing function is to our data structure, the runtime of this function will have a significant effect on the overall runtime of our data structure. This means we want our hash code to be “easily” computable (ideally constant time), so that we may maintain the O(1) runtime characteristic that makes hashing so special and efficient!

24.4 Handling Collisions: Linear Probing and External Chaining

In hashing, a collision occurs when we have multiple elements that have the same index in our array.

There are two common methods to deal with collisions in hash tables:

1. **Linear Probing:** Store the colliding keys elsewhere in the array, potentially in the next open array space. This method can be seen with distributed hash tables, which you will see in later computer science courses that you may take.
2. **External Chaining:** A simpler solution is to store all the keys with the same hash value together in a collection of their own, such as a `LinkedList`. This collection of entries sharing a single index is called a **bucket**.

24.5 Resizing & Hash Table Performance

No matter how good our `hashCode()` method is, if the underlying array of our hash table is small and we add a lot of keys to it, then we will start getting more and more collisions. Because of this, a hash table should expand its underlying array once it starts to fill up (much like how an `ArrayList` expands once it fills up).

To keep track of how full our hash table is, we define the term **load factor**, which is the ratio of the number of elements inserted over the total physical length of the array.

$$\text{load factor} = \frac{\texttt{size()}}{\texttt{array.length}}$$

A very important equation

For our hash table, we will define the maximum load factor that we will allow. **If adding another key-value pair would cause the load factor to exceed the specified maximum load factor, then the hash table should resize.** This is usually done by doubling the underlying array length. Java's default maximum load factor is 0.75 which provides a good balance between a reasonably-sized array and reducing collisions.

Note that if we are trying to add a key-value pair and the key already exists in the hash map, the corresponding value should be updated but no resizing should occur.

As an example, let's consider what happens if our hash table has an array length of 10 and currently contains 7 elements. Each of these 7 elements are hashed modulo 10 because we want to get an index within the range of 0 through 9. The current load factor is 7/10, or 0.7, just under the threshold.

If we try to insert one more element, we would have a total of 8 elements in our hash table and a load factor of 0.8. Because this would cause the load factor to exceed the maximum load factor, we must resize the underlying array to length 20 before we insert the element. Remember that since our procedure for locating an entry in the hash table is to take the `hashCode() % array.length` and our array's length has changed from 10 to 20, all the elements in the hash table need to be relocated. Once all the elements have been relocated and our new element has been added, we will have a load factor of 8/20, or 0.4, which is below the maximum load factor.

24.6 Summary

In this chapter, we learned about hashing, a powerful technique for turning a more complex object like a `String` into a numerically representable value like an `int`.

The *hash table* is a data structure that combines the *hash function* with the fact that arrays can be indexed in constant time. Using the hash table and the map abstract data type, we can build a `HashMap` which allows for amortized constant time access to any key-value pair so long as we know which bucket the key falls into.

However, we quickly demonstrated that this naive implementation has several drawbacks: the ability to represent all different kinds of objects, memory efficiency, and collisions.

We investigated the importance of the `hashCode()` function to gain an understanding of how it affects the runtime of the hash table. To allow for smaller `size()` in the array, we used the modulo operator to shrink hash values down to a specified range of numbers. We then added external chaining to solve collisions by allowing multiple entries to live in a single bucket in the form of a LinkedList, and explored resizing functionality based on the load factor!

24.7 Exercises

1. Potpourri

a. True or False: Resizes are triggered if adding a key-value pair causes the load factor to be greater than or equal to the specified maximum load factor.

▶ Answer to Q1(a)

b. True or False: The `hashCode()` function can have varied return types.

▶ Answer to Q1(b)

2. hashCode()….

In order for a hash code to be valid, objects that are equivalent to each other (i.e. .equals() returns true) must return equivalent hash codes. If an object does not explicitly override the `hashCode()` method, it will inherit the `hashCode()` method defined in the Object class, which returns the object's address in memory.

Here are four potential implementations of Integer's `hashCode()` function. Assume that `intValue()` returns the value represented by the Integer object.

Categorize each `hashCode()` implementation as either a valid or an invalid hash function. If it is valid, point out a flaw or disadvantage. If it is invalid, explain why.

Question (a)

```
public int hashCode() {
    return -1;
}
```

▶ Answer for Q2(a)

Question (b)

```
public int hashCode() {
    return intValue() * intValue();
}
```

▶ Answer for Q2(b)

Question (c)

```
public int hashCode() {
    Random rand = new Random();
    return rand.nextInt();
}
```

▶ Answer for Q2(c)

Question (d)

```
public int hashCode() {
    return super.hashCode();
}
```

▶ Answer for Q2(d)

3. Hashin' and Resizin'

Given the provided `hashCode()` implementation, hash the items listed below with external chaining (the first item is already inserted for you). Assume the load factor is 1, and the initial underlying array has size of 4. Use geometric resizing with a resize factor of 2. You may draw more boxes to extend the array when you need to resize.

```
/** Returns 0 if word begins with 'a', 1 if it begins with 'b', etc. */
public int hashCode() {
    return word.charAt(0) - 'a';
}
```

```
["apple", "cherry", "fig", "guava", "durian", "apricot", "banana"]
```

▶ Answer for Question 3

24. Hashing I